

Four-Bit Full Adder & Four-Bit Full Subtractor design

CENG 2112.01 Lab for Digital Circuits

Submitted by

David Dulaney

2344357

Computer Science

University of Houston-Clear Lake

Houston, Texas 77058

Apr 14,2026

Contents

Abstract	2
Introduction	2
Requirements	3
Prelab	4
Implementation	5
Design Code/Design Diagrams	6
Schematics	12
Testbench	13
Waveform/Results	17
Conclusion	25

Abstract

A full adder is a combinational circuit that requires three inputs and will produce two outputs. The first two inputs are normal inputs (A and B), the third input is a carry (Cin). The normal output is the SUM, which is the normal sum of A and B. The other output (Cout) is a carry output which transfers over if a 0 or a 1 is being carried over. For example $1 + 1 + 1$ will be a SUM of 1 and have a Cout as 1. A full subtractor is similar to a full adder but instead of a cout and sum, the outputs are diff and bout which means when multiple inputs happen, for example $1\ 0\ 1$ and they get subtracted. The diff would output 0 and the bout would also be 0 since a 1 didn't have to be carried to execute the subtraction.

Introduction

A full adder (#7483) is a device that when given three inputs, it will produce two outputs, with one being the sum of the inputs and the other being the carry value of the inputs (indicated by a 0 or 1). The purpose of the full adder is to perform binary addition and carrying the output

through multiple adders to produce a carry and sum output. This is used in calculators to be able to do complex calculations through binary addition. In order to understand how full adders work, minterm or maxterm expansion is used to get the Boolean expression of the full adder, it then can be simplified to conclude the most efficient circuit. A full subtractor (#74283) is a device when given three inputs, it will produce two outputs, with one being diff and the other being bout which is the bit needed to be carried to execute the subtraction (indicated by a 0 or 1). In order to understand how full subtractors work, minterm or maxterm expansion is used to get the Boolean expression of the full subtractor, it then can be simplified to conclude the most efficient circuit.

Requirements

The requirement for this lab is to simulate a full adder and full subtractor adding or subtracting respectively and display that on an FPGA board. We used a logic analyzer within Multisim to check the waveforms with different inputs of 1s and 0s and built a schematic as well as testing their behavior in a simulation. Then we programmed an FPGA board using the seven-segment display to display the addition and subtraction of the full adder and full subtractor respectively.

Zn-lab 7

Full adder truth table:

A	B	B'	D	Count
0	0	1	0	0
0	0	1	1	1
0	1	0	1	1
0	1	0	0	0
1	0	0	1	1
1	0	0	0	0
1	1	0	0	0
1	1	1	1	1

$D = A \oplus B \oplus B'$
 $Count = AB + AB' + BB'$
 $Count = A(B + B') + AB$

4-bit subtractor
 $D_n = A_n \oplus B_n$

Diagram of a 4-bit subtractor using full adders:

Inputs: A_1, B_1 (for F0), A_2, B_2 (for F1), A_3, B_3 (for F2), A_4, B_4 (for F3).
 Outputs: D_1, D_0 (from F0), D_2, D_1 (from F1), D_3, D_2 (from F2), D_4, D_3 (from F3).
 Carry/Count outputs: $Count_1$ (from F0), $Count_2$ (from F1), $Count_3$ (from F2), $Count_4$ (from F3).

Full Adder truth table:

A	B	Ch	Sum	count
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

$Sum = A \oplus B \oplus Ch$
 $Count = AB + ACh + BCh$
 $factored\ count = AB + Ch(A \oplus B)$

4

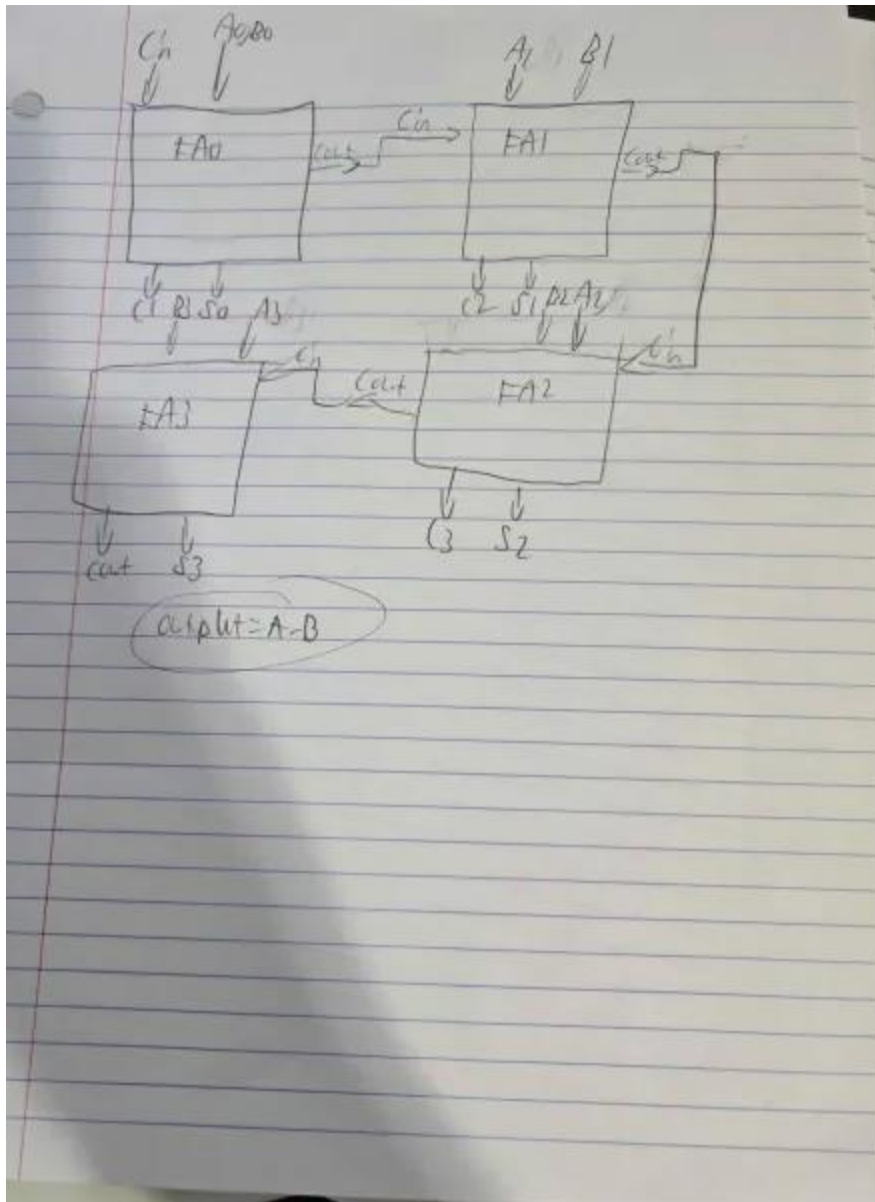


figure 1.2: prelab design for a four bit full adder

Implementation

The first step in implementation was to build a schematic in VHDL to simulate what the full adder/ full subtractor is doing as well as testing their behaviors in a test bench. After that, we programmed an FPGA board to display in a seven-segment system, what the output of each addition/subtraction is.

Design Code/Design Diagrams

DM code: for full adder

-- Company:

-- Engineer:

--

-- Create Date: 03/24/2026 02:06:56 PM

-- Design Name:

-- Module Name: Project_7_Full_Adder - Behavioral

-- Project Name:

-- Target Devices:

-- Tool Versions:

-- Description:

--

-- Dependencies:

--

-- Revision:

-- Revision 0.01 - File Created

-- Additional Comments:

--

library IEEE;

use IEEE.STD_LOGIC_1164.ALL;

```
-- Uncomment the following library declaration if using
-- arithmetic functions with Signed or Unsigned values
--use IEEE.NUMERIC_STD.ALL;

-- Uncomment the following library declaration if instantiating
-- any Xilinx leaf cells in this code.
--library UNISIM;
--use UNISIM.VComponents.all;
```

```
entity Project_7_Full_Adder is
```

```
    Port ( X : in STD_LOGIC;
           Y : in STD_LOGIC;
           Zin : in STD_LOGIC;
           subtraction : buffer STD_LOGIC;
           InternalCout : out STD_LOGIC);
```

```
end Project_7_Full_Adder;
```

```
architecture Behavioral of Project_7_Full_Adder is
```

```
begin
```

```
subtraction <= ((x xor y) xor zin);
```

```
InternalCout<=((((not X) and Zin) or ((not X) and Y))or(Y and Zin));
```

```
end Behavioral;
```

Dm code for full adder:

-- Company:

-- Engineer:

--

-- Create Date: 02/24/2026 02:43:25 PM

-- Design Name:

-- Module Name: project_5 - Behavioral

-- Project Name:

-- Target Devices:

-- Tool Versions:

-- Description:

--

-- Dependencies:

--

-- Revision:

-- Revision 0.01 - File Created

-- Additional Comments:

--

library IEEE;

use IEEE.STD_LOGIC_1164.ALL;

-- Uncomment the following library declaration if using


```

-- arithmetic functions with Signed or Unsigned values
--use IEEE.NUMERIC_STD.ALL;

-- Uncomment the following library declaration if instantiating
-- any Xilinx leaf cells in this code.
--library UNISIM;
--use UNISIM.VComponents.all;
--instantiating the ports with input and output logic
entity project_5 is
    Port ( A : in STD_LOGIC;
           B : in STD_LOGIC;
           Bin : in STD_LOGIC;
           Diff : out STD_LOGIC;
           Bout : out STD_LOGIC);
end project_5;

architecture Behavioral of project_5 is

begin
--inserted logic box for diff and bout kmaps equations
Diff<=A xor B xor Bin;
Bout<= (A and B) or (B and Bin) or (Bin and A);

```

end Behavioral;

Dm code for full subtractor:

-- Company:

-- Engineer:

--

-- Create Date: 02/24/2026 02:43:25 PM

-- Design Name:

-- Module Name: project_5 - Behavioral

-- Project Name:

-- Target Devices:

-- Tool Versions:

-- Description:

--

-- Dependencies:

--

-- Revision:

-- Revision 0.01 - File Created

-- Additional Comments:

--

library IEEE;

use IEEE.STD_LOGIC_1164.ALL;

-- Uncomment the following library declaration if using

```

-- arithmetic functions with Signed or Unsigned values
--use IEEE.NUMERIC_STD.ALL;

-- Uncomment the following library declaration if instantiating
-- any Xilinx leaf cells in this code.
--library UNISIM;
--use UNISIM.VComponents.all;
--instantiating the ports with input and output logic
entity project_5 is
    Port ( A : in STD_LOGIC;
          B : in STD_LOGIC;
          Bin : in STD_LOGIC;
          Diff : out STD_LOGIC;
          Bout : out STD_LOGIC);
end project_5;

architecture Behavioral of project_5 is

begin
--inserted logic box for diff and bout kmaps equations
Diff<=A xor B xor Bin;
Bout<= (not A and Bin) or (B and Bin) or (not A and B);

end Behavioral;

```

Schematics/Implemented design

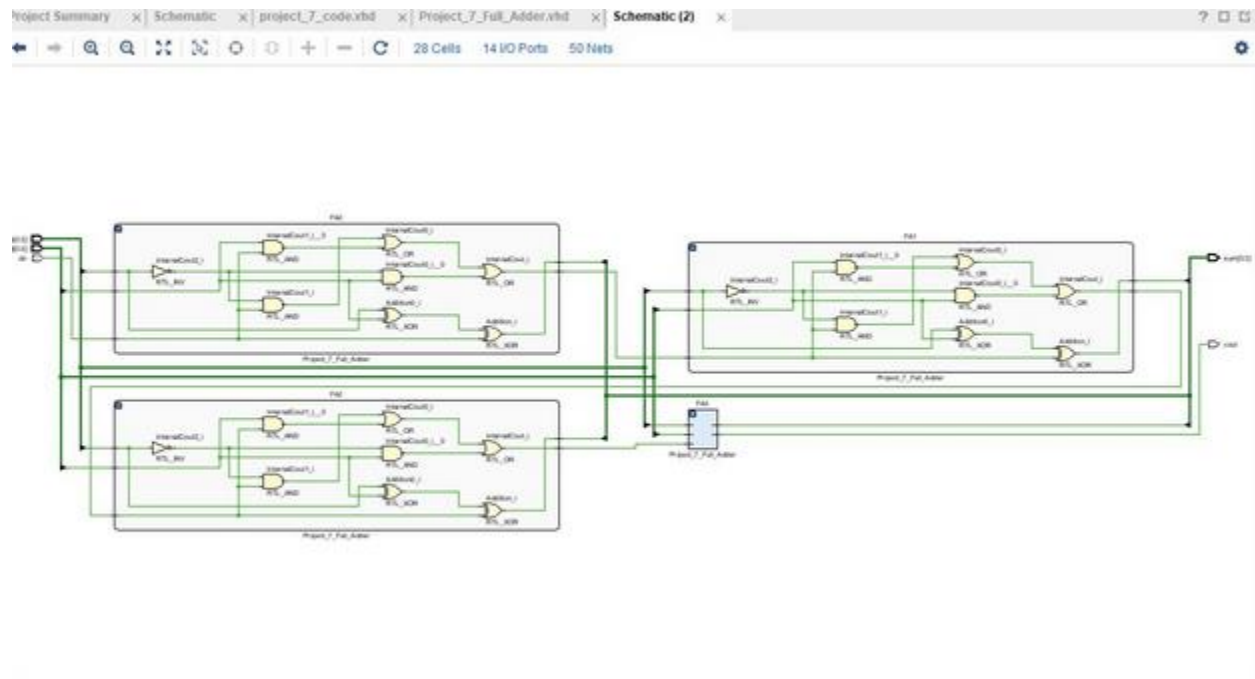


Figure 2.1: Schematic of the four bit full adder

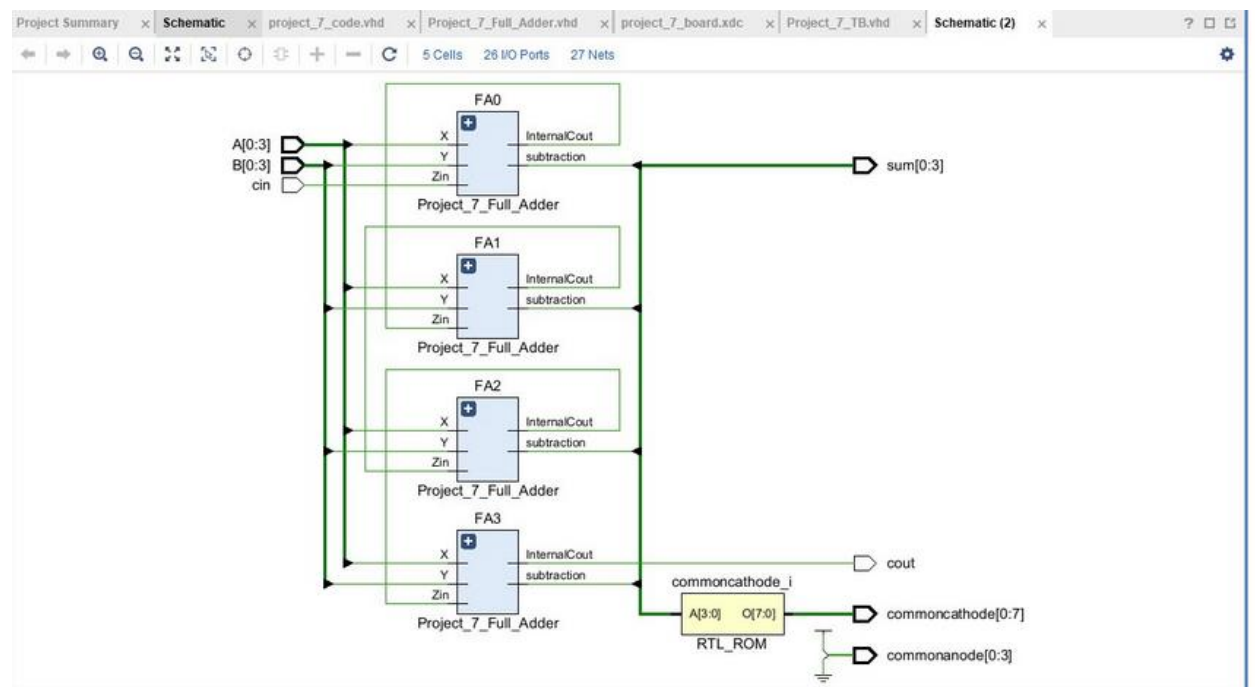


Figure 2.2: Schematic of the four bit full subtractor

Testbench

TB code for full adder:

-- Company:

-- Engineer:

--

-- Create Date: 03/24/2026 02:06:56 PM

-- Design Name:

-- Module Name: Project_7_Full_Adder - Behavioral

-- Project Name:

-- Target Devices:

-- Tool Versions:

-- Description:

--

-- Dependencies:

--

-- Revision:

-- Revision 0.01 - File Created

-- Additional Comments:

--

library IEEE;

use IEEE.STD_LOGIC_1164.ALL;

```
-- Uncomment the following library declaration if using
-- arithmetic functions with Signed or Unsigned values
--use IEEE.NUMERIC_STD.ALL;

-- Uncomment the following library declaration if instantiating
-- any Xilinx leaf cells in this code.
--library UNISIM;
--use UNISIM.VComponents.all;
```

entity Project_7_Full_Adder is

```
    Port ( X : in STD_LOGIC;
           Y : in STD_LOGIC;
           Zin : in STD_LOGIC;
           Addition : buffer STD_LOGIC;
           InternalCout : out STD_LOGIC);
```

end Project_7_Full_Adder;

architecture Behavioral of Project_7_Full_Adder is

begin

```
Addition <= (x xor y) xor zin;
```

```
InternalCout<=((not X and Zin) or (Y and Zin) )or (not X and Y);
```

ARDUINO BOARD code for full adder:

```
#inputs
```

```
set_property PACKAGE_PIN V2 [get_ports {A[0]}]
```

```

set_property IOSTANDARD LVCMOS33 [get_ports {A[0]}]
set_property PACKAGE_PIN W13 [get_ports {A[1]}]
set_property IOSTANDARD LVCMOS33 [get_ports {A[1]}]
set_property PACKAGE_PIN W14 [get_ports {A[2]}]
set_property IOSTANDARD LVCMOS33 [get_ports {A[2]}]
set_property PACKAGE_PIN V15 [get_ports {A[3]}]
set_property IOSTANDARD LVCMOS33 [get_ports {A[3]}]
set_property PACKAGE_PIN W15 [get_ports {B[0]}]
set_property IOSTANDARD LVCMOS33 [get_ports {B[0]}]
set_property PACKAGE_PIN W17 [get_ports {B[1]}]
set_property IOSTANDARD LVCMOS33 [get_ports {B[1]}]
set_property PACKAGE_PIN W16 [get_ports {B[2]}]
set_property IOSTANDARD LVCMOS33 [get_ports {B[2]}]
set_property PACKAGE_PIN V16 [get_ports {B[3]}]
set_property IOSTANDARD LVCMOS33 [get_ports {B[3]}]
set_property PACKAGE_PIN V17 [get_ports {cin}]
set_property IOSTANDARD LVCMOS33 [get_ports {cin}]

```

#lights

```

set_property PACKAGE_PIN L1 [get_ports {sum[0]}]
set_property IOSTANDARD LVCMOS33 [get_ports {sum[0]}]
set_property PACKAGE_PIN P1 [get_ports {sum[1]}]
set_property IOSTANDARD LVCMOS33 [get_ports {sum[1]}]
set_property PACKAGE_PIN N3 [get_ports {sum[2]}]
set_property IOSTANDARD LVCMOS33 [get_ports {sum[2]}]
set_property PACKAGE_PIN P3 [get_ports {sum[3]}]

```

```

set_property IOSTANDARD LVCMOS33 [get_ports {sum[3]}]
set_property PACKAGE_PIN U3 [get_ports {cout}]
set_property IOSTANDARD LVCMOS33 [get_ports {cout}]

set_property PACKAGE_PIN W7 [get_ports {commoncathode[0]}]
set_property IOSTANDARD LVCMOS33 [get_ports {commoncathode[0]}]
set_property PACKAGE_PIN W6 [get_ports {commoncathode[1]}]
set_property IOSTANDARD LVCMOS33 [get_ports {commoncathode[1]}]
set_property PACKAGE_PIN U8 [get_ports {commoncathode[2]}]
set_property IOSTANDARD LVCMOS33 [get_ports {commoncathode[2]}]
set_property PACKAGE_PIN V8 [get_ports {commoncathode[3]}]
set_property IOSTANDARD LVCMOS33 [get_ports {commoncathode[3]}]

set_property PACKAGE_PIN U5 [get_ports {commoncathode[4]}]
set_property IOSTANDARD LVCMOS33 [get_ports {commoncathode[4]}]
set_property PACKAGE_PIN V5 [get_ports {commoncathode[5]}]
set_property IOSTANDARD LVCMOS33 [get_ports {commoncathode[5]}]
set_property PACKAGE_PIN U7 [get_ports {commoncathode[6]}]
set_property IOSTANDARD LVCMOS33 [get_ports {commoncathode[6]}]
set_property PACKAGE_PIN V7 [get_ports {commoncathode[7]}]
set_property IOSTANDARD LVCMOS33 [get_ports {commoncathode[7]}]

set_property PACKAGE_PIN W4 [get_ports {commonanode[0]}]
set_property IOSTANDARD LVCMOS33 [get_ports {commonanode[0]}]

```



```

set_property PACKAGE_PIN V4 [get_ports {commonanode[1]}]

    set_property IOSTANDARD LVCMOS33 [get_ports {commonanode[1]}]

set_property PACKAGE_PIN U4 [get_ports {commonanode[2]}]

    set_property IOSTANDARD LVCMOS33 [get_ports {commonanode[2]}]

set_property PACKAGE_PIN U2 [get_ports {commonanode[3]}]

    set_property IOSTANDARD LVCMOS33 [get_ports {commonanode[3]}]

```

Waveform/Results

The behavioral simulation as shown in figure 3 shows how a four bit full adder adds two binary bits together with a carry in and a carryout bit to correctly output the addition.

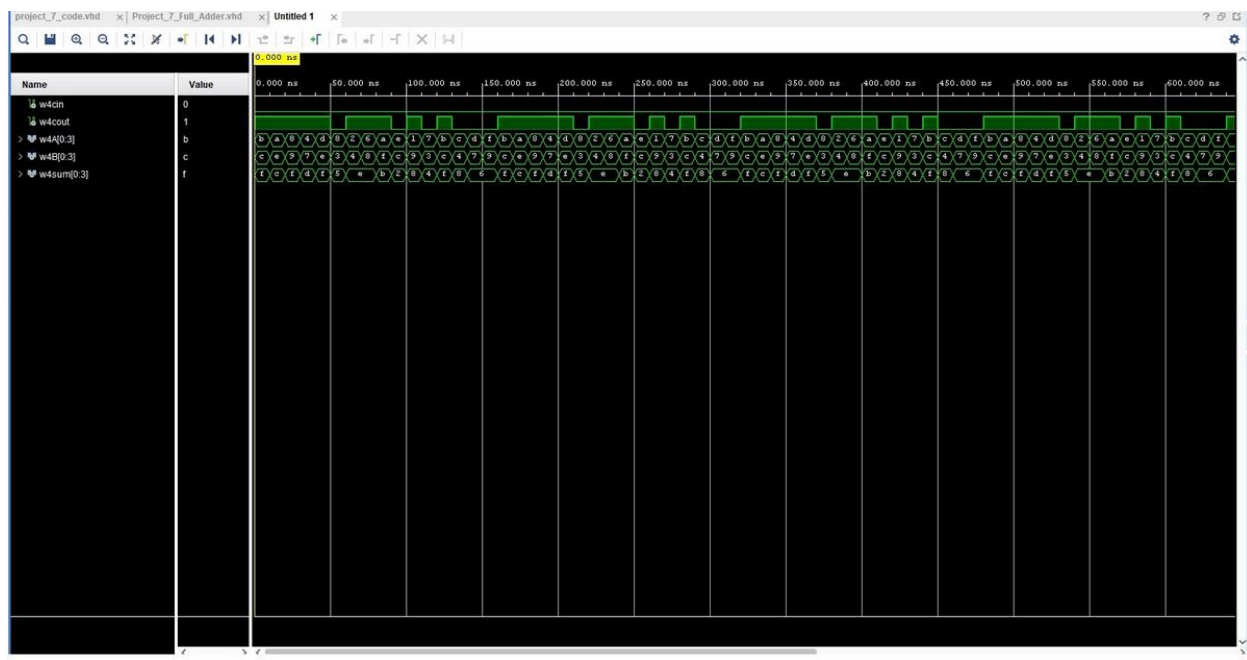


figure 3: behavioral simulation of a full adder.

The result in figure 4.1 shows the seven-segment display of the FPGA board displaying a 4 after adding bits 2 and 2.

The result in figure 4.2 shows the seven-segment display of the FPGA board displaying a F after adding bits 8 and 7.

The result in figure 4.3 shows the seven-segment display of the FPGA board displaying a 0 after adding bits 0 and 0

The result in figure 4.4 shows the seven-system display of the FPGA board displaying a 1 after adding bits 1 and 0.

The result in figure 4.5 shows the seven-system display of the FPGA board displaying a 2 after adding bits 1 and 1.

The result in figure 4.6 shows the seven-system display of the FPGA board displaying a 2 after subtracting bits 4 and 2.

The result in figure 4.7 shows the seven-system display of the FPGA board displaying a 4 after subtracting bits 4 and 1.

The result in figure 4.8 shows the seven-system display of the FPGA board displaying a 7 after subtracting bits 8 and 1.

The result in figure 4.9 shows the seven-system display of the FPGA board displaying a 6 after subtracting bits 8 and 2.

The result in figure 4.10 shows the seven-system display of the FPGA board displaying a 4 after subtracting bits 8 and 4.

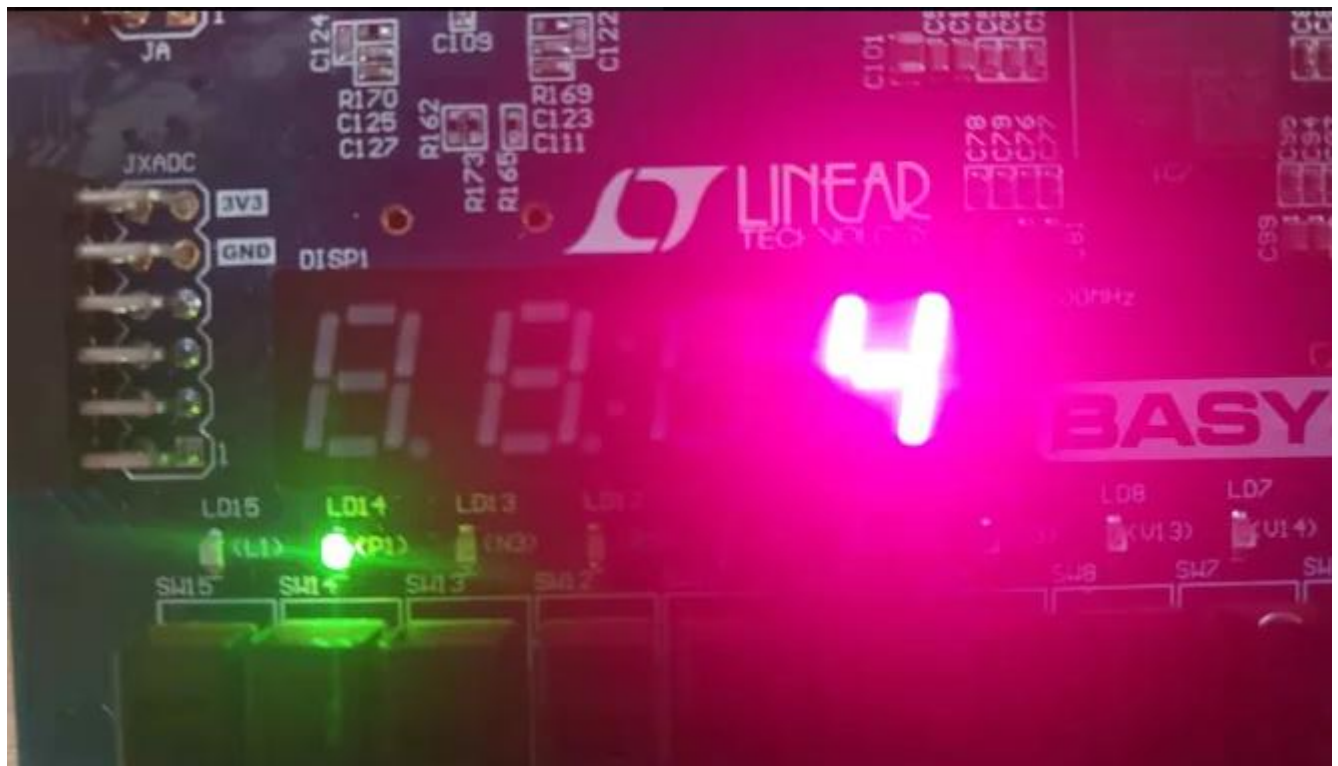


figure 4.1: seven-segment display showing a 4 when binary bits 2 and 2 are added.



figure 4.2: seven-segment display showing a F when binary bits 8 and 7 are added.



figure 4.3: seven-segment display showing a 0 when binary bits 0 and 0 are added.

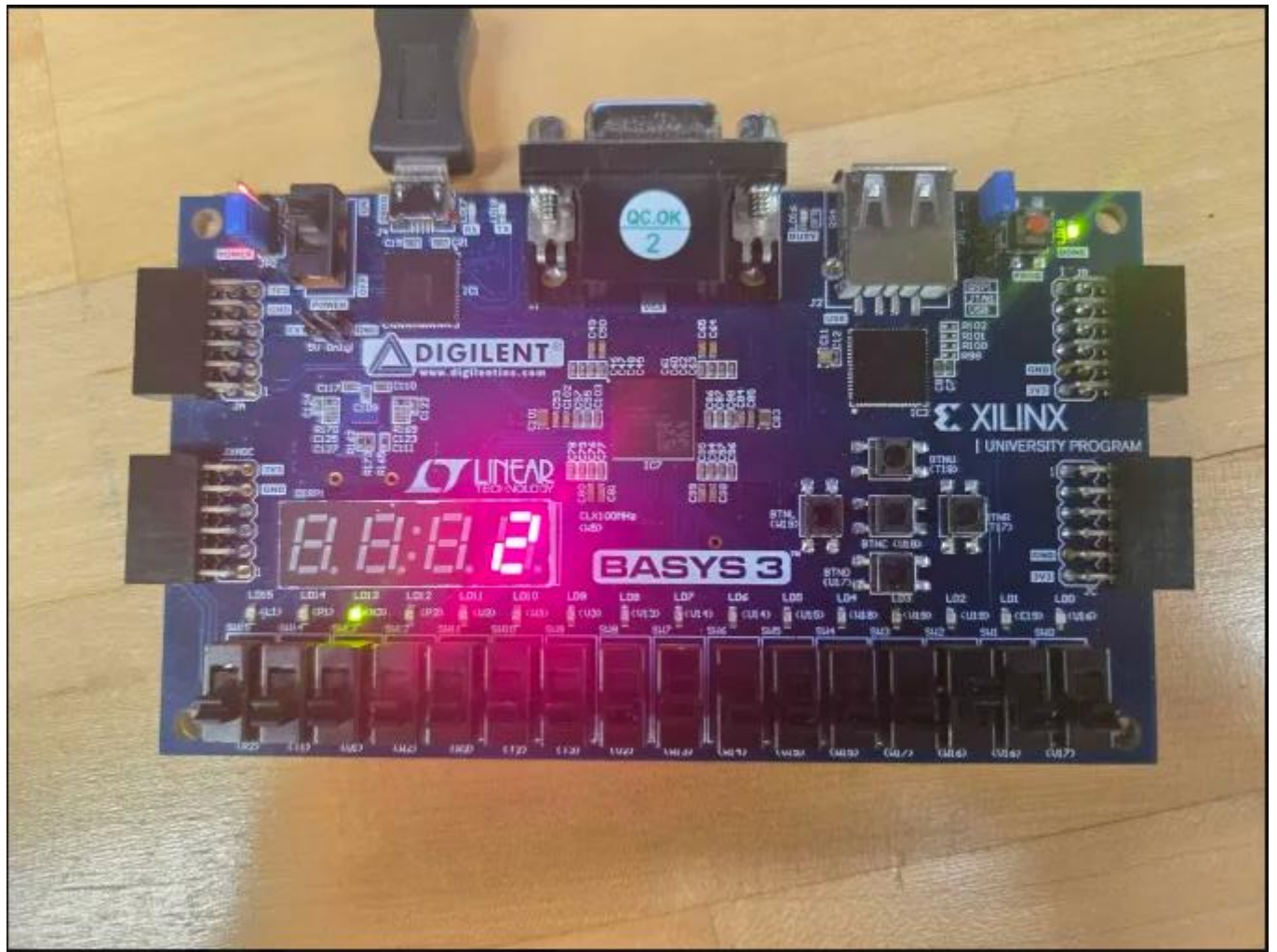


figure 4.6: seven-segment display showing a 2 when binary bits 4 and 2 are subtracted.

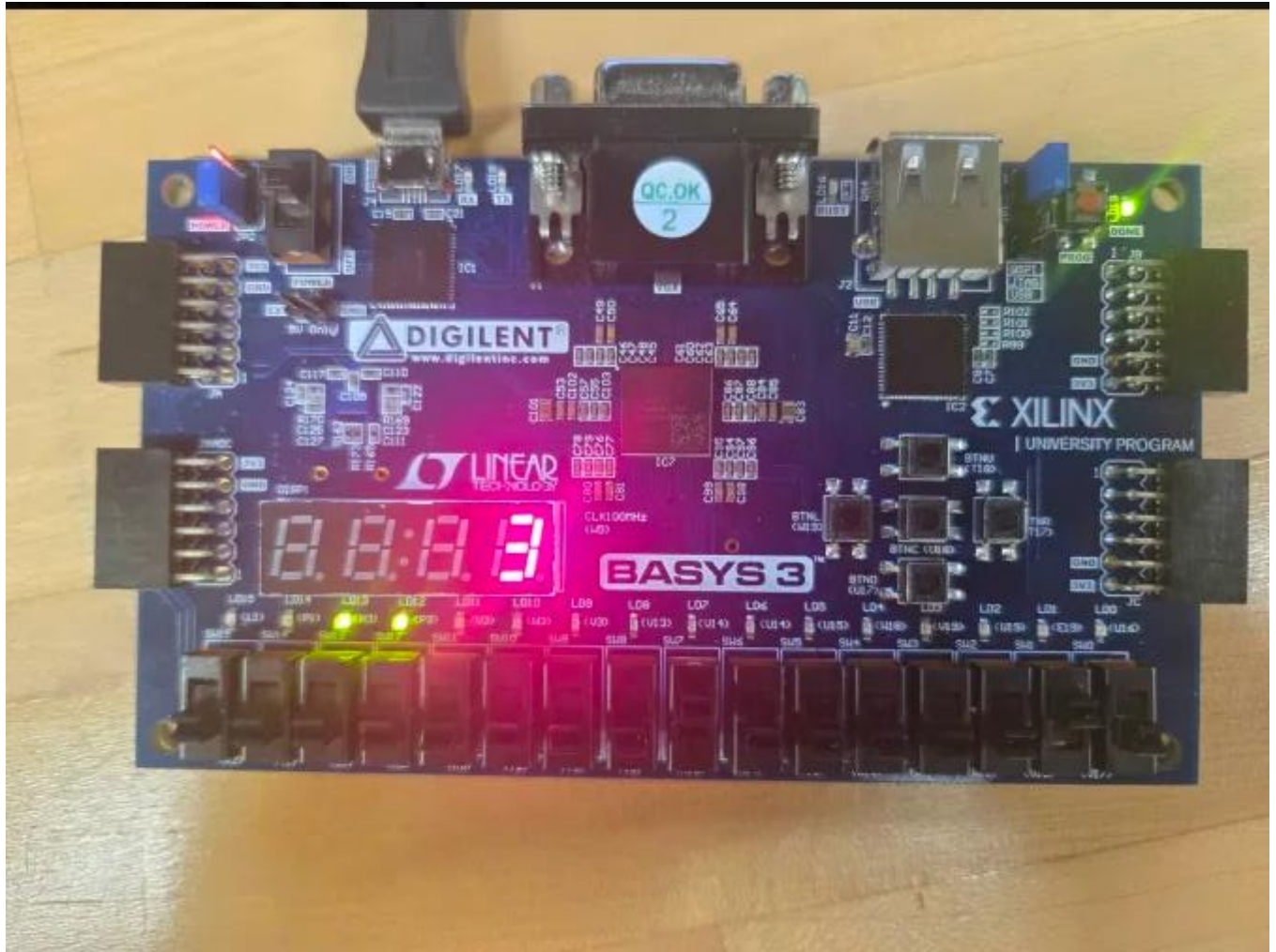


figure 4.7: seven-segment display showing a 3 when binary bits 4 and 1 are subtracted.

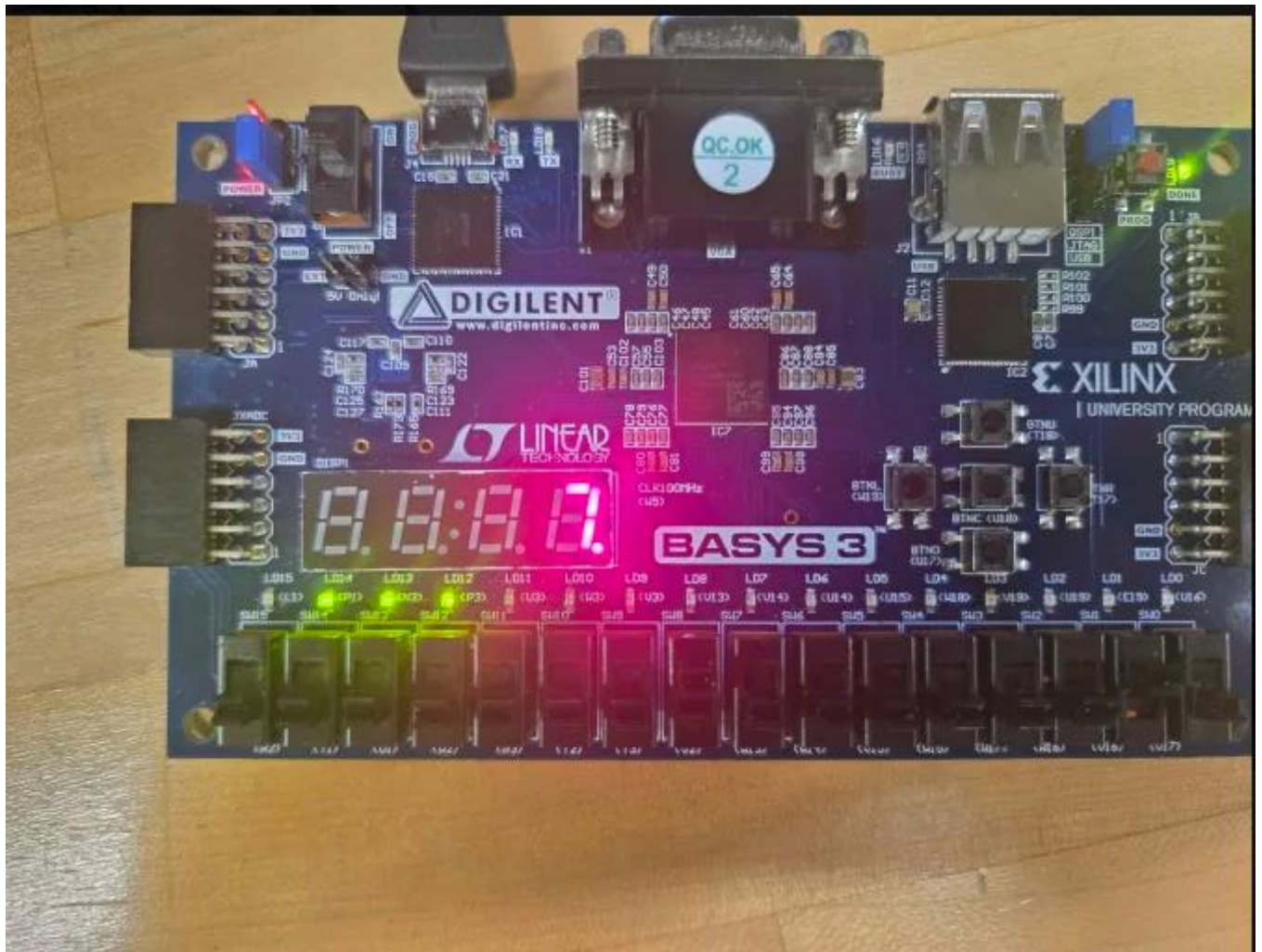


figure 4.8: seven-segment display showing a 7 when binary bits 8 and 1 are subtracted.

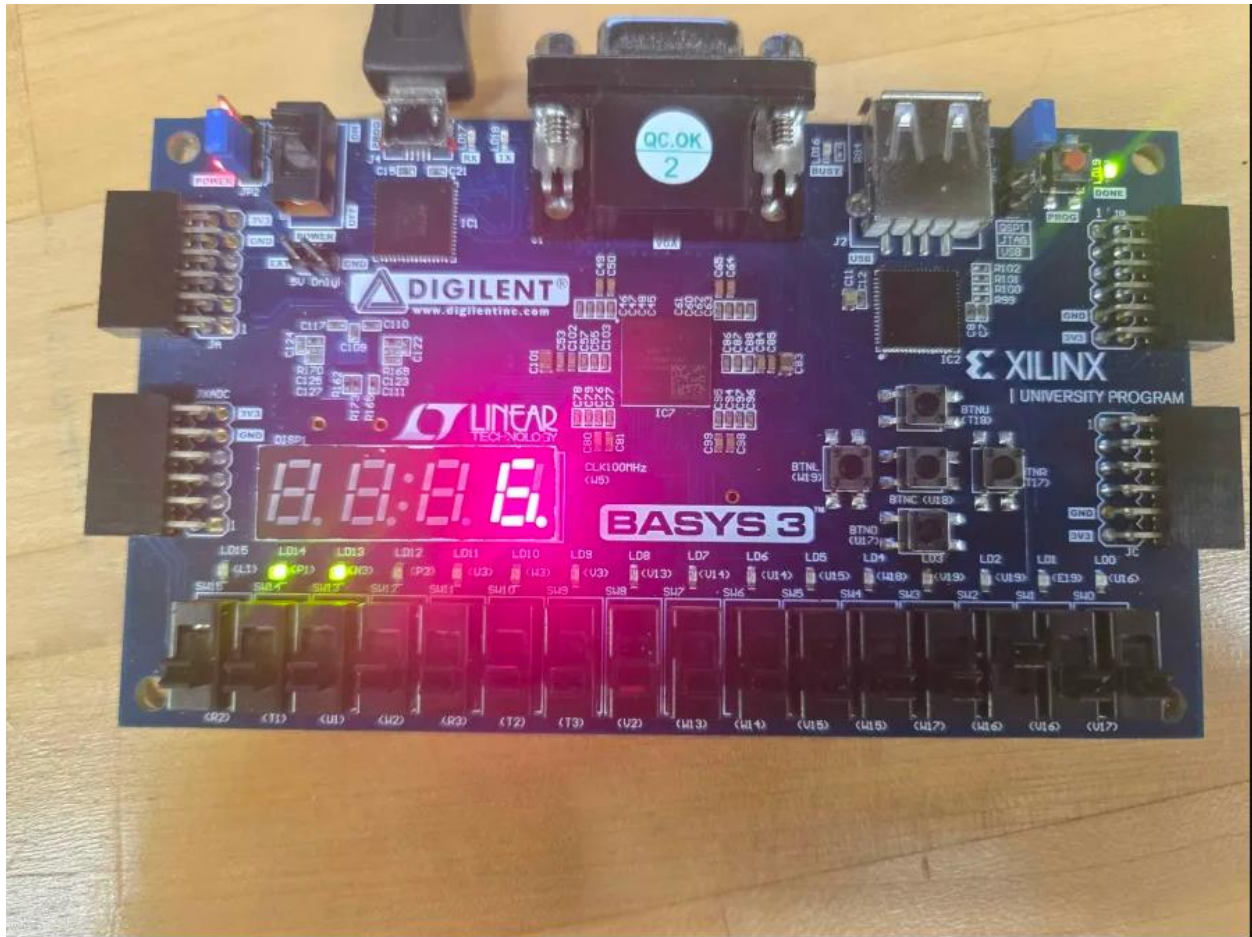


figure 4.9: seven-segment display showing a 6 when binary bits 4 and 2 are subtracted.

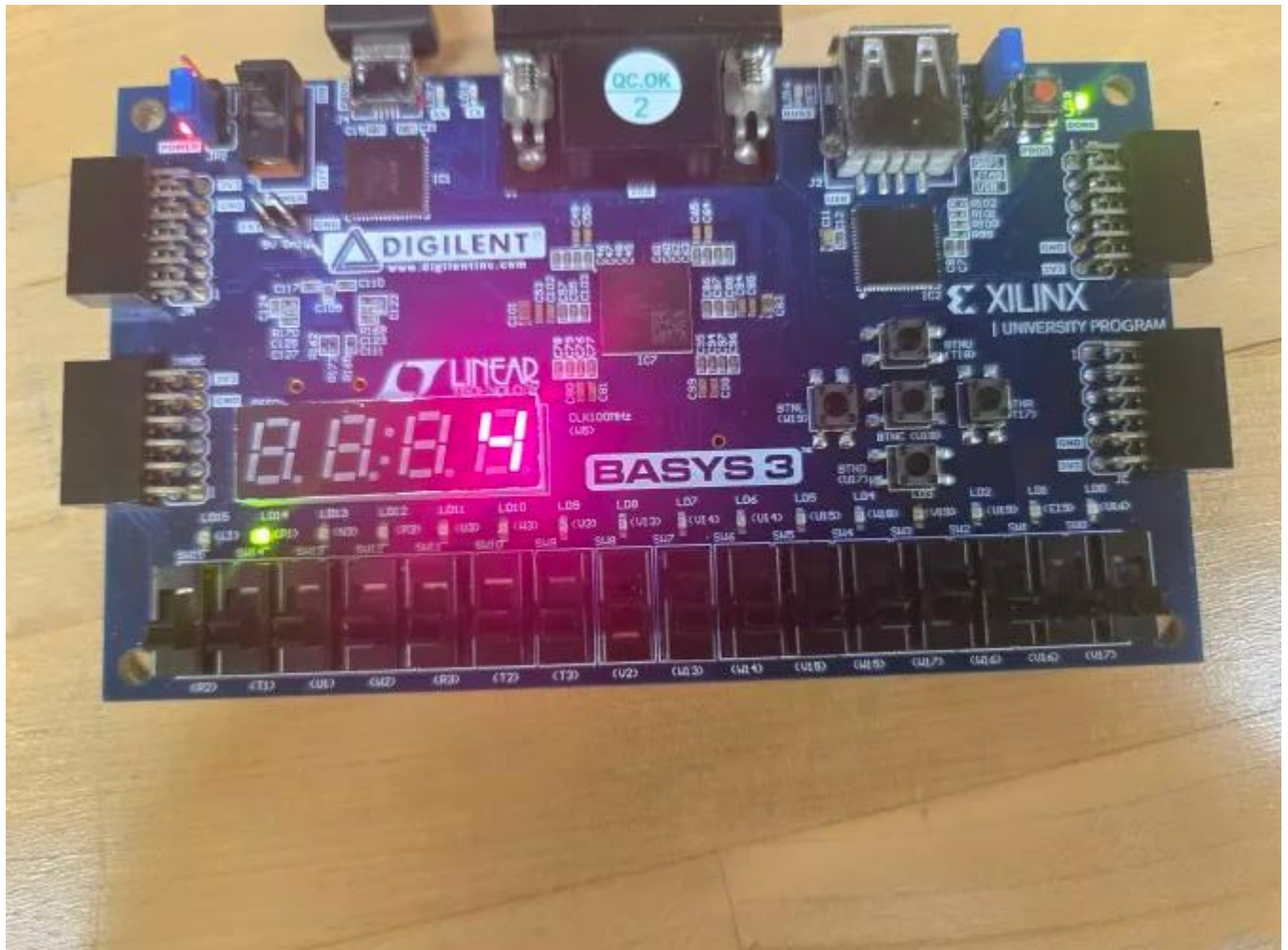


figure 4.10: seven-segment display showing a 6 when binary bits 8 and 4 are subtracted.

Conclusion

This project was designed to understand that the how a full adder and full subtractor works and their differences of carrying out a 1 or borrowing a 1 to and from the next bit respectively. This project also helped understand how a seven-segment display is programmed and how it can be used to translate the output of binary addition and subtraction to a human that is not knowledgeable in this field.